Universidad Politécnica de Chiapas

Ingeniería · Minería de Datos · 2026A

Integración de motores de búsqueda

Tarea 4, Corte 2

BM25, embeddings y búsqueda híbrida en un sistema fullstack,
con corpus obtenido por web scraping y prueba de carga de hasta 50 usuarios

Alumno: Bruno Blanco Lucio
Matrícula: 233275
Docente: Ramses Alejandro Camas Najera
Fecha: 5 de noviembre de 2026

Repositorio: `https://github.com/BrunoBlanco011/BLANCO_LUCIO_BRUNO_233275_C2_T4_INTEGRACION`

Resumen

Se integraron los tres motores de búsqueda de la unidad en un sistema fullstack: **BM25** (Lab 3), **embeddings** `paraphrase-multilingual-mpnet-base-v2` (Tarea 2) e **híbrido** RRF + cross-encoder (Tarea 3). El backend es una API REST en FastAPI y el frontend es una página HTML/JS que permite buscar con un motor o comparar los tres lado a lado. El corpus se construyó por **web scraping** de 60 artículos de Wikipedia en español sobre Chiapas (23 872 palabras; de 140 a 629 por documento), y las fuentes están en el Anexo A.

Con 19 consultas evaluadas a mano, BM25 y el híbrido empatan en nDCG@5 (**0.922**), y los embeddings quedan en 0.757. Este resultado es el inverso del de la Tarea 3: en artículos largos, las paráfrasis casi siempre comparten palabras con el documento. En la prueba de carga (de 1 a 50 usuarios concurrentes, 5 600 peticiones, 0 errores), la latencia p50/p95 con 1 usuario fue de **12/17 ms** (BM25), **25/35 ms** (embeddings) y **93/103 ms** (híbrido). Con 50 usuarios subió a **259/305 ms**, **958/1 106 ms** y **3 671/3 742 ms**: los tres motores se saturan desde los 5 usuarios, en unas 190, 50 y 13 peticiones por segundo. Con estos resultados se eligió BM25 como motor por omisión y el híbrido como modo de "búsqueda profunda".

Índice

1. Corpus: web scraping de Wikipedia **3**
1.1. Cómo se obtuvo . . . 3
1.2. Qué contiene . . . 3

2. Los tres motores **4**

3. El sistema fullstack **5**
3.1. Arquitectura . . . 5

4. Calidad de los resultados **6**

5. Desempeño: latencia con hasta 50 usuarios concurrentes **8**
5.1. Cómo se midió . . . 8
5.2. Resultados . . . 9
5.3. Comportamiento de los motores . . . 10

6. Criterio de decisión **12**
6.1. Calidad contra costo . . . 12
6.2. El criterio . . . 12
6.3. ¿Cuándo usar cada motor según el corpus? . . . 12

7. Ventajas y desventajas de cada motor **13**

8. Conclusiones **15**

A. Fuentes del corpus **15**

B. Reproducir los resultados **17**

1. Corpus: web scraping de Wikipedia

1.1. Cómo se obtuvo

El corpus se construyó con `scraping/scrape_wikipedia.py`. El script descarga el **HTML** de cada artículo de `https://es.wikipedia.org` con `requests` y lo procesa con `BeautifulSoup`; no usa la API de texto plano. Por cada página:

1. Toma el título de `#firstHeading` y el número de revisión (`wgRevisionId`), para que el corpus se pueda reproducir.
2. Elimina tablas, infoboxes, llamadas a nota (`[1]`, `[cita requerida]`), botones de edición y estilos.
3. Recorre los párrafos `<p>` del cuerpo (excepto los que están dentro de tablas, listas o figuras) y descarta los de menos de 8 palabras.
4. Junta párrafos completos hasta llegar a unas 350 palabras y descarta el documento si no llega a 100.

El HTML se guarda en caché local, de modo que cada página se pide una sola vez, y entre peticiones se espera 1 s. Se identifica al cliente con un *User-Agent* genérico del proyecto.

Hubo que corregir tres cosas durante el scraping:

- El primer intento falló por certificados SSL en Windows; se resolvió con `truststore`, que usa los certificados del sistema.
- El HTML actual de Wikipedia (Parsoid) envuelve cada sección en `<section>`. Como el parser buscaba solo los `<p>` hijos directos, sacaba 0 palabras de cada página.
- Dos títulos apuntaban a otra página: *Palenque* es el artículo sobre la palabra y *Zoques* es una desambiguación. Se reemplazaron por *Zona arqueológica de Palenque* y *Zoque (etnia)*. Además, 5 títulos no existían y se cambiaron por otros del mismo tema.

1.2. Qué contiene

Se eligieron temas de Chiapas para seguir con el dominio de los laboratorios anteriores (las 14 noticias del Lab 1), pero con documentos mucho más largos. El resultado son **60 documentos** con un total de **23 872 palabras** (entre 140 y 629 por documento, con media de 398), en seis categorías (Tabla 1). La lista completa de fuentes, con URL y revisión, está en el Anexo A y en `data/fuentes.csv`. Los textos son de Wikipedia, con licencia CC BY-SA 4.0.

Tabla 1: Composición del corpus.

Categoría	Docs	Palabras (media)	Ejemplos
turismo	15	401	Cañón del Sumidero, Palenque, Bonampak, Tuxtla Gutiérrez
naturaleza	13	399	Selva Lacandona, El Chichón, quetzal, jaguar, río Grijalva
cultura	12	395	Parachicos, marimba, pueblo tsotsil, EZLN, Rosario Castellanos
tecnología	8	416	UNACH, inteligencia artificial, minería de datos, robótica
riesgos	6	386	Terremoto de 2017, huracán Stan, dengue, deforestación
economía	6	380	Café de Chiapas, cacao, presa La Angostura, Tren Maya
Total	**60**	**398**	

2. Los tres motores

Los tres motores son los de las tareas anteriores, adaptados a este corpus y con la misma interfaz: `buscar(consulta, k)`, que devuelve los pares (documento, score) y el tiempo de cada etapa. Están en `backend/motores.py`.

1. BM25 (Lab 3).
Preprocesamiento del Lab 7: minúsculas, sin URLs ni HTML, lemas de spaCy (`es_core_news_sm`) sin acentos y sin stopwords, pero conservando las negaciones. Usa $k_1 = 1.5$ y $b = 0.75$. A diferencia de los laboratorios, que recorrían todo el corpus, aquí se usa un **índice invertido** (término → documentos), así que solo se calculan los documentos que comparten algún término con la consulta. Si ninguno comparte términos, devuelve una lista vacía.

2. Embeddings (Tarea 2).
Encoder `paraphrase-multilingual-mpnet-base-v2` (768 dimensiones), con la limpieza de la Tarea 2, que conserva mayúsculas, acentos y stopwords. Hubo un cambio de diseño: en la Tarea 2 las notas tenían unas 40 palabras, pero este encoder solo lee **128 tokens**, y los documentos ahora tienen cerca de 400 palabras. Si se codifica el documento entero, el encoder ve solo el primer párrafo. Por eso cada documento se parte en **pasajes de 110 palabras con traslape de 30** (308 pasajes en total), cada pasaje lleva el título del artículo, y el score del documento es el **coseno del mejor pasaje**.

3. Híbrido (Tarea 3).
Es la arquitectura recomendada en la Tarea 3: **RRF** ($k = 60$) entre el top-20 de BM25 y el top-20 de los embeddings, seguido de un **re-ranking** del top-10 con el cross-encoder `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`. Al principio el cross-encoder recibía el mejor pasaje de cada documento, y así reordenaba mal: para *temblor muy fuerte...* subía *Selva Lacandona* al primer lugar, porque un pasaje cortado a la mitad pierde el contexto del artículo. Ahora recibe **título + documento completo**, truncado a 512 tokens; con ese cambio, el score del terremoto pasó de -5.85 a -1.75 y quedó en el primer lugar.

3. El sistema fullstack

3.1. Arquitectura

```
Navegador (frontend/: HTML + CSS + JS, sin dependencias)
    |  fetch  GET /api/buscar?q=...&motor=...   GET /api/comparar?q=...
    v
FastAPI + Uvicorn (backend/app.py, 1 proceso)
    |  endpoints "def" -> pool de hilos (40)
    |  middleware: cabecera X-Tiempo-Servidor-ms
    v
backend/motores.py
    MotorBM25  : spaCy (lock) + indice invertido ..... CPU
    MotorDenso : mpnet (lock) + matriz 308 x 768 ..... GPU
    MotorHibrido: BM25 + Denso -> RRF -> cross-encoder (lock)  GPU
    ^
    | se construye una vez al arrancar (lifespan) + calentamiento
data/corpus.json  <-  scraping/scrape_wikipedia.py  <-  es.wikipedia.org
```

Figura 1: Arquitectura del sistema.

Backend. Es una API REST con FastAPI (Tabla 2). Los motores se construyen **una sola vez**, en el *lifespan* de la aplicación: cargar spaCy toma 13.9 s, cargar mpnet y codificar los 308 pasajes 26.5 s, y cargar el cross-encoder 3.8 s. Al terminar se hace una búsqueda de calentamiento con cada motor, porque la primera llamada a la GPU es lenta. Los endpoints son funciones síncronas (`def`), así que FastAPI los ejecuta en su pool de hilos y una búsqueda lenta no bloquea a las demás. Cada modelo tiene un candado (`threading.Lock`): varias peticiones pueden avanzar a la vez, pero solo una a la vez usa la GPU (o spaCy). Esta decisión es la que explica cómo crece la latencia con la concurrencia (Sección 5).

Tabla 2: Endpoints de la API. La documentación interactiva está en `/docs`.

Ruta	Qué devuelve
`GET /api/buscar?q=&motor=&k=`	top-k de un motor, con score, fragmento, URL de la fuente, latencia total y por etapa
`GET /api/comparar?q=&k=`	los tres motores con la misma consulta
`GET /api/documento/{id}`	texto completo y fuente
`GET /api/corpus`	lista de documentos y estadísticas
`GET /api/salud`	estado, dispositivo (cuda/cpu), número de pasajes y tiempos de carga

Frontend. Es una sola página en HTML, CSS y JavaScript, sin *frameworks*, servida por el mismo FastAPI. Permite buscar con un motor o **comparar los tres en columnas**. Cada columna muestra la latencia del motor con una barra que la divide por etapas (BM25, embeddings, RRF, cross-encoder) y resalta en los fragmentos las palabras de la consulta. Al hacer clic en un resultado se abre el documento completo con el enlace a Wikipedia. También muestra el tiempo de ida y vuelta medido en el navegador y el que reporta el servidor (Figura 2).

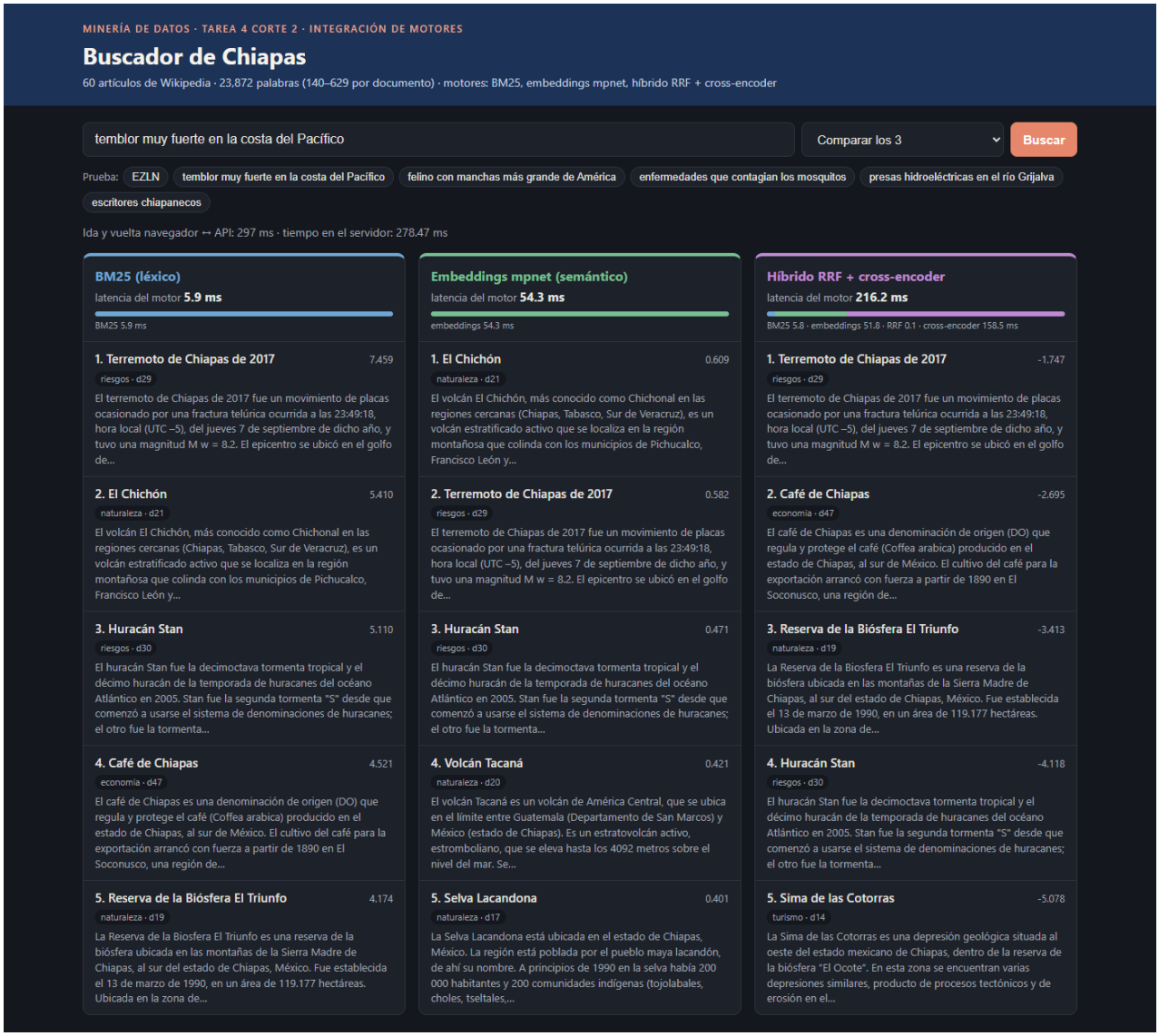


Figura 2: Frontend en modo "comparar los 3" con la consulta *temblor muy fuerte en la costa del Pacífico*. BM25 y el híbrido ponen el terremoto en primer lugar; los embeddings ponen primero al volcán El Chichón. Los tiempos son de una sola consulta, tomada mientras arrancaba el navegador de la captura; las latencias del reporte salen de la prueba de carga.

Entorno. Windows 11, Python 3.12, NVIDIA RTX 4060 Laptop (8 GB), `torch` 2.6 (CUDA 12.4), `sentence-transformers` 6.2, `spaCy` 3.8, FastAPI 0.115 y Uvicorn 0.34 (1 proceso).

4. Calidad de los resultados

La latencia solo tiene sentido junto con la calidad, así que antes de la prueba de carga se midió qué tan buenos son los resultados. Se escribieron **19 consultas** con relevancia graduada (3 = el documento trata el tema, 2 = lo trata en parte, 1 = lo menciona), juzgadas leyendo el texto de cada documento (`evaluacion/qrels.json`). Se repartieron igual que en el Lab 7:

- **Léxicas (6):** siglas y nombres propios: *EZLN*, *UNACH*, *Montes Azules*, *Tren Maya*, *Coffea*

arabica denominación de origen y *Bonampak y Yaxchilán*.

- **Semánticas (6):** paráfrasis que evitan la palabra clave del documento, como *temblor muy fuerte en la costa del Pacífico* (el documento dice *terremoto*), *felino con manchas más grande del continente* (*jaguar*) o *aguardiente artesanal usado en rituales indígenas* (*pox*).
- **Mixtas (7):** temas con varios documentos relevantes, como *zonas arqueológicas mayas en la selva* o *presas hidroeléctricas en el río Grijalva*.

Se usaron las métricas del Lab 7 (nDCG@5 con ganancia $2^{rel} - 1$, MRR y R@5) más P@5. Como ablación se agregó el híbrido *sin* cross-encoder (solo RRF), para separar lo que aporta cada etapa (`evaluacion/evaluar_calidad.py`).

Tabla 3: Calidad sobre las 19 consultas. Las tres últimas columnas son el nDCG@5 por tipo de consulta.

Motor	nDCG@5	MRR	R@5	P@5	Léxicas	Semánt.	Mixtas
BM25	**0.922**	**0.961**	**0.886**	**0.379**	**0.977**	0.846	**0.940**
Embeddings (mpnet)	0.757	0.803	0.789	0.347	0.785	0.617	0.852
Híbrido, solo RRF	0.856	0.868	0.868	0.368	0.851	0.770	0.934
Híbrido RRF + cross-encoder	**0.922**	**0.961**	0.838	0.358	0.963	**0.868**	0.932

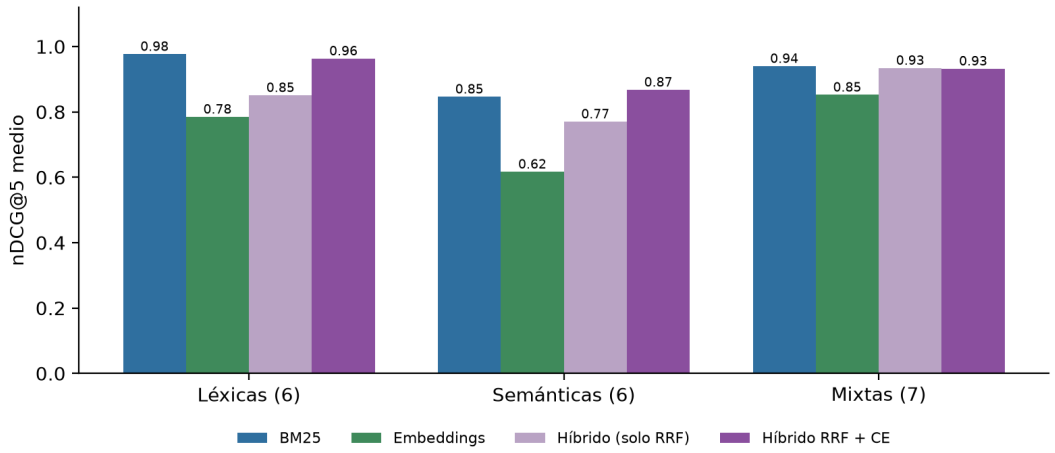


Figura 3: nDCG@5 medio por tipo de consulta.

El resultado no es el de la Tarea 3. Con las 14 notas cortas, BM25 valía 0 en las consultas semánticas y el denso las resolvía todas. En este corpus pasa lo contrario: **BM25 empata con el híbrido** (nDCG@5 = 0.922) y le saca 0.165 a los embeddings. Las causas son tres:

1. **Los documentos largos casi siempre contienen la paráfrasis.** Un artículo de 400 palabras usa muchas palabras para hablar de su tema. Al pasar las consultas "semánticas" por el preprocesamiento de BM25, casi todas comparten lemas con su documento relevante: *temblor…* comparte *fuerte*, *costa* y *pacífico* con el artículo del terremoto; *aguardiente…* comparte *aguardiente*, *artesanal*, *ritual* e *indígena* con el del pox; *enfermedades que contagian los mosquitos* comparte *enfermedad* y *mosquito* con el del dengue. En las notas de 40 palabras del Lab 7 esa coincidencia no existía. La única consulta sin vocabulario en común es la del jaguar (solo comparte *continente*), y es justo donde BM25 cae a 0.431.

2. **Los embeddings encuentran el tema, pero no el documento exacto.** Los errores del denso son documentos "vecinos": para *temblor* pone primero *El Chichón* (el volcán, cuyo artículo habla de sismos); para el felino pone primero al cocodrilo; para el ave de plumaje verde pone a la guacamaya antes que al quetzal; para *aguardiente...* devuelve cinco artículos sobre pueblos indígenas y deja fuera al del pox (nDCG = 0). En las léxicas falla con nombres raros: en *Montes Azules* pone primero *Cascadas de Agua Azul* (0.500), y en *Coffea arabica* pone primero *Soconusco* (0.431).

3. **El encoder lee 128 tokens.** El score de cada documento depende de cómo se parte en pasajes. La ablación (Tabla 4) muestra que con el mismo encoder el nDCG@5 va de 0.739 a 0.824 según la representación del documento. Promediar los 2 mejores pasajes o usar el embedding medio rinde más que el máximo que usa el motor. No se cambió el motor porque la variante se elegiría con las mismas 19 consultas con las que se evalúa, que es el mismo sesgo que la Tarea 3 encontró con el α óptimo. Aun la mejor variante (0.824) queda por debajo de BM25.

Tabla 4: Ablación del motor denso: cinco formas de representar un documento de 400 palabras con un encoder de 128 tokens (nDCG@5).

Representación del documento	Global	Léxicas	Semánt.	Mixtas
Máximo sobre pasajes (**el motor**)	0.757	0.785	0.617	0.852
Media de los 2 mejores pasajes	**0.824**	0.786	0.768	**0.903**
Embedding medio de los pasajes	0.814	0.665	**0.902**	0.867
Solo el primer pasaje (el resumen del artículo)	0.739	**0.818**	0.650	0.748
Documento completo truncado a 128 tokens	0.739	**0.818**	0.650	0.748

Qué aporta cada etapa del híbrido. RRF solo (0.856) queda *entre* los dos motores: arregla lo peor del denso (*aguardiente* pasa de 0 a 0.631 y *Montes Azules* de 0.500 a 0.631), pero cuando el denso mete documentos equivocados en su top-20, RRF los mezcla con los buenos de BM25. El cross-encoder es lo que lo pone a la par de BM25: sube *zonas arqueológicas mayas en la selva* a 0.986 (BM25: 0.759), *aguardiente* y *Coffea arabica* a 1.000, y obtiene el mejor nDCG@5 en las semánticas (0.868). A cambio, baja respecto a RRF en cuatro mixtas (*pueblos indígenas de los Altos*, de 0.884 a 0.752; *bebidas de maíz y cacao*, de 1.000 a 0.947; *escritores chiapanecos*, de 1.000 a 0.920; *ave de plumaje verde*, de 1.000 a 0.983), y ningún sistema resuelve la del jaguar (0.431 en BM25 y en el híbrido).

Con 19 consultas, una diferencia de 0.05 en nDCG equivale a mover uno o dos documentos en una o dos consultas. Las diferencias claras son las grandes: embeddings $\ll$ RRF $<$ BM25 $\approx$ híbrido con cross-encoder.

5. Desempeño: latencia con hasta 50 usuarios concurrentes

5.1. Cómo se midió

La prueba está en `carga/prueba_carga.py`. Cada **usuario virtual** es una corrutina de `asyncio` que, con `httpx`, manda una consulta a `/api/buscar`, espera la respuesta y manda la siguiente, sin pausa entre consultas. Es un modelo de carga *cerrado* y es el peor caso: un usuario real tarda unos segundos en leer los resultados. Las consultas se toman al azar de un conjunto de 31 (las 19 de evaluación y 12 más), con semilla fija.

- **Niveles:** 1, 5, 10, 25 y 50 usuarios concurrentes para cada motor. Cada nivel manda al menos 300 peticiones (500 con 50 usuarios) después de 5 de calentamiento que no se miden. En total fueron **5 100 peticiones** en los niveles y 500 en el escenario mixto.
- **Escenario mixto:** 50 usuarios en los que cada consulta elige motor al azar. Es lo más parecido al uso real del frontend.
- **Latencia:** se mide en el cliente (ida y vuelta HTTP completa) y de ahí salen p50, p95 y p99. También se guarda el tiempo que reporta el servidor en la cabecera `X-Tiempo-Servidor-ms`.
- **Entorno:** servidor y cliente corren en la misma laptop (sin red de por medio), con Uvicorn en 1 proceso y modelos en la GPU. Las 5 600 peticiones respondieron **200 OK**, sin errores.

5.2. Resultados

Tabla 5: Latencia en el cliente (ms) y rendimiento por motor y número de usuarios concurrentes. "Servidor" es la mediana del tiempo dentro de FastAPI.

Motor	Usuarios	**p50**	**p95**	p99	Servidor p50	Peticiones/s
BM25	1	**11.8**	**17.0**	27.3	4.3	80.3
	5	25.2	30.3	41.0	16.7	193.8
	10	50.0	57.6	64.5	41.5	194.1
	25	126.4	138.9	167.7	116.7	192.5
	50	**258.5**	**305.2**	379.1	248.7	188.0
Embeddings	1	**24.5**	**35.2**	40.7	16.8	38.0
	5	101.2	134.7	137.1	90.1	46.7
	10	190.9	209.9	219.5	180.1	51.8
	25	490.9	595.5	606.3	478.6	48.7
	50	**957.9**	**1 105.9**	1 126.0	947.6	50.3
Híbrido	1	**93.2**	**103.3**	108.3	84.8	10.7
	5	372.8	412.5	473.5	361.7	13.2
	10	742.2	787.5	809.3	730.8	13.4
	25	1 851.3	1 936.8	1 966.2	1 839.2	13.4
	50	**3 670.8**	**3 742.1**	3 754.1	3 659.4	13.5
Mixto (1/3 cada uno)	50	2 030.3	2 499.8	2 570.4	2 007.9	30.2

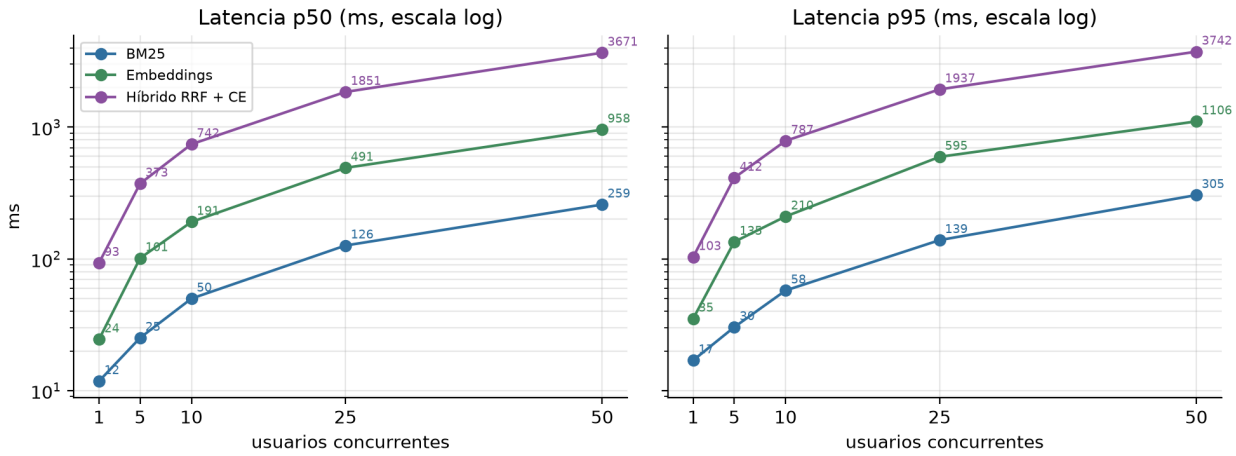


Figura 4: p50 y p95 contra usuarios concurrentes (escala logarítmica).

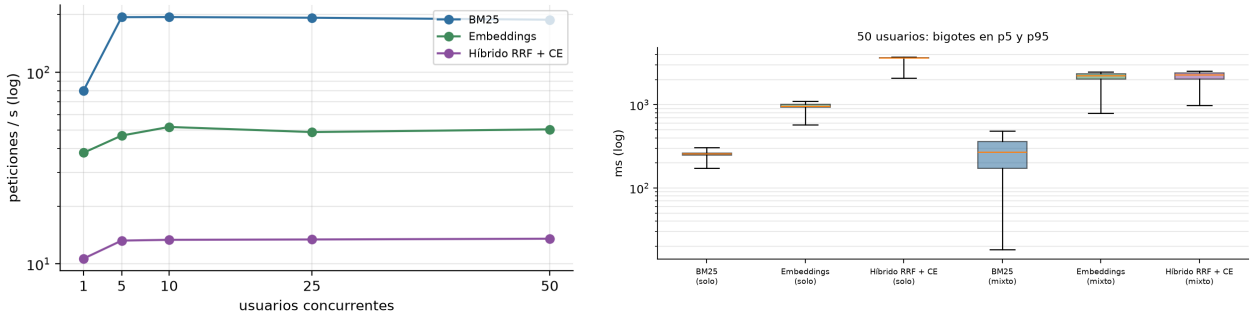


Figura 5: Izquierda: rendimiento (peticiones por segundo). Derecha: distribución de la latencia con 50 usuarios, cada motor solo y en el escenario mixto.

5.3. Comportamiento de los motores

1. **Con un usuario, el costo sigue el número de modelos.** BM25 tarda 4.3 ms en el servidor, porque solo lematiza la consulta y recorre el índice invertido. Los embeddings tardan 16.8 ms: codifican la consulta en la GPU y hacen un producto matriz–vector de 308×768. El híbrido tarda 84.8 ms, y más de dos tercios de ese tiempo (60 a 90 ms, según la consulta) se van en el cross-encoder, que lee 10 pares de hasta 512 tokens. Los 7 a 9 ms de diferencia entre cliente y servidor son el costo de HTTP en local.

2. **Los tres se saturan muy pronto, y después la latencia crece en línea recta.** El rendimiento deja de subir a partir de 5 usuarios: unas **190 peticiones/s** para BM25, **50** para los embeddings y **13.4** para el híbrido. A partir de ahí, agregar usuarios solo alarga la cola, y la latencia sigue la ley de Little (latencia $\approx$ usuarios/rendimiento): con 50 usuarios, $50/188 = 266$ ms para BM25 (medido: 259), $50/50.3 = 994$ ms para los embeddings (958) y $50/13.5 = 3.7$ s para el híbrido (3.67 s). Prácticamente todo el tiempo es espera: la mediana dentro del servidor es casi igual a la del cliente.

3. **La causa es que el servidor procesa una consulta a la vez.** El rendimiento máximo coincide con el inverso del tiempo de una consulta ($1/4.3$ ms ≈ 230, $1/16.8$ ms ≈ 60, $1/84.8$ ms ≈ 12).

Es decir, aunque FastAPI tenga 40 hilos, las peticiones avanzan de una en una: BM25 por el GIL de Python y el candado de spaCy, y los otros dos por el candado de la GPU. Se eligió así a propósito, porque usar los modelos desde varios hilos sin candado no es seguro. El precio es que el servidor no escala con la concurrencia.

4. **p95 queda muy cerca de p50.** Con 50 usuarios, p95/p50 es 1.18 en BM25, 1.15 en embeddings y 1.02 en el híbrido. Como todas las peticiones pasan por la misma cola, la variación entre consultas (consultas más largas, más candidatos) se diluye en el tiempo de espera. La excepción es BM25 con 1 usuario: su p99 (27 ms) es 2.3 veces su p50. Con tiempos tan cortos, cualquier pausa del sistema operativo o de Python pesa mucho (no se investigó la causa exacta).

5. **En el escenario mixto, el motor lento arrastra a los demás.** Con 50 usuarios repartidos entre los tres motores, BM25 sigue rápido (p50 = 269 ms, p95 = 487 ms), porque usa la CPU y no compite por la GPU. Los embeddings, en cambio, *empeoran* respecto a su escenario solo: p50 de 2 245 ms contra 958 ms. Las búsquedas densas comparten el encoder con la etapa densa del híbrido, y la GPU se reparte con el cross-encoder, así que quedan formadas detrás de las consultas híbridas. El rendimiento total (30.2 peticiones/s) es el promedio ponderado de los tres.

¿Cuántos usuarios aguanta cada motor? Si se toma como meta un p95 menor a 500 ms (un valor común para que una búsqueda se sienta inmediata), BM25 la cumple con los 50 usuarios (305 ms), los embeddings hasta unos 20 (p95 = 210 ms con 10 y 596 ms con 25) y el híbrido solo hasta unos 5 (413 ms). Con usuarios reales que leen los resultados unos 10 s entre consultas, cada uno manda unas 0.1 consultas por segundo, así que el mismo servidor atendería a unos 1 900 usuarios con BM25, 500 con embeddings y 130 con el híbrido.

Cómo se podría escalar. Por orden de impacto sobre el cuello de botella, que es la cola de la GPU:

- **Agrupar consultas** (*dynamic batching*): juntar en un solo lote de GPU las consultas que llegan en una ventana de pocos ms. Codificar 16 consultas cuesta casi lo mismo que codificar una.
- **Cache** de las consultas frecuentes (LRU), que en un buscador real se repiten mucho.
- **Menos candidatos para el cross-encoder** (5 en lugar de 10) o un cross-encoder más chico o cuantizado (ONNX, int8).
- **Varios procesos de Uvicorn** para BM25, que usa la CPU, y separar el motor denso y el híbrido en un servicio de inferencia propio.

6. Criterio de decisión

6.1. Calidad contra costo

Tabla 6: Los tres motores lado a lado. Latencias medidas en el cliente.

	BM25	Embeddings	Híbrido RRF + CE
nDCG@5 (19 consultas)	**0.922**	0.757	**0.922**
nDCG@5 semánticas	0.846	0.617	**0.868**
p50 / p95 con 1 usuario (ms)	**12 / 17**	25 / 35	93 / 103
p50 / p95 con 50 usuarios (ms)	**259 / 305**	958 / 1 106	3 671 / 3 742
Rendimiento máximo (peticiones/s)	≈**190**	≈50	≈13
Usuarios con p95 < 500 ms	**50+**	≈20	≈5
Tiempo de arranque	14 s (spaCy)	27 s	+4 s sobre los otros dos
Necesita GPU	no	recomendable	recomendable
Devuelve "sin resultados"	sí	no (siempre da k)	no

6.2. El criterio

La elección se hizo en este orden:

1. **Calidad mínima aceptable.** Se descarta el motor que se equivoca en el tema. Los embeddings solos quedan fuera en este corpus: su nDCG@5 (0.757) está 0.165 por debajo de los otros dos y fallan en nombres propios (*Montes Azules* → *Agua Azul*).

2. **Latencia bajo la carga esperada.** Entre los motores con buena calidad, se elige el que cumple la meta de p95 con la concurrencia esperada. BM25 y el híbrido empatan en nDCG@5, pero BM25 es 8 veces más rápido con un usuario y 14 veces más rápido con 50.

3. **El tipo de consulta.** El híbrido solo vale lo que cuesta si los usuarios escriben consultas sin el vocabulario del documento. Es el único que mejora en las semánticas (0.868 frente a 0.846 de BM25) y corrige casos que BM25 deja mal, como *zonas arqueológicas mayas en la selva* (0.986 frente a 0.759).

Decisión para este sistema: BM25 como motor por omisión y el **híbrido como modo "búsqueda profunda"** que el usuario activa, o que se activa solo cuando BM25 devuelve pocos resultados o scores bajos. Así, casi todas las consultas cuestan 12 ms, y el cross-encoder se usa solo donde puede ayudar. Si hubiera que elegir *un solo* motor para 50 usuarios simultáneos en este hardware, sería BM25: es el único que cumple p95 < 500 ms sin perder calidad. El híbrido sería la mejor opción si el tráfico fuera bajo (menos de 5 usuarios a la vez) o si se le agregara *batching* en la GPU.

6.3. ¿Cuándo usar cada motor según el corpus?

La comparación con la Tarea 3 muestra que el mejor motor **depende del corpus**, no solo del algoritmo:

Tabla 7: El mismo par de motores en los dos corpus de la unidad.

	14 noticias cortas (Labs 1–7)	60 artículos de Wikipedia (esta tarea)
Largo	≈40 palabras	≈400 palabras
Vocabulario	breve, una sola forma de decir cada cosa	extenso; repite el tema con sinónimos
BM25 nDCG@5	0.735 (0 en semánticas)	0.922 (0.846 en semánticas)
Denso nDCG@5	0.918	0.757
Mejor opción	denso o híbrido	BM25 o híbrido

- **BM25** conviene en corpus de **documentos largos y bien redactados** (enciclopedias, manuales, documentación técnica, expedientes), donde el texto contiene muchas formas de nombrar su tema. También cuando las consultas son **nombres propios, siglas, códigos o números** (*EZLN*, *Coffea arabica*, un número de expediente), cuando hace falta poder decir "no hay resultados" y cuando no hay GPU o se esperan muchos usuarios.
- **Los embeddings** convienen en corpus de **textos cortos** (tuits, títulos, preguntas frecuentes, reseñas, mensajes de soporte), donde la consulta y el documento rara vez comparten palabras. También para buscar entre idiomas (el encoder es multilingüe), para recomendar "documentos parecidos" y como primera etapa de un sistema RAG. En documentos largos hay que cuidar el troceo en pasajes, porque el encoder solo lee 128 tokens.
- **El híbrido** conviene cuando **no se sabe de antemano** qué tipo de consulta llegará, o cuando el usuario mezcla nombres con descripciones (*zonas arqueológicas mayas en la selva*), y cuando la precisión del primer resultado importa más que la latencia: búsqueda legal o médica, o el recuperador de un RAG, donde el modelo generativo tarda segundos de todos modos. Su costo crece con el número de candidatos que se re-ordenan, no con el tamaño del corpus.

7. Ventajas y desventajas de cada motor

BM25 (léxico)

Ventajas

- El más rápido y el que mejor escala: 12 ms con 1 usuario, 259 ms con 50 y unas 190 peticiones/s, sin GPU.
- Tuvo la mejor calidad global en este corpus (nDCG@5 = 0.922, MRR = 0.961) y es casi perfecto con nombres propios y siglas (0.977).
- Es explicable: el score se puede descomponer término por término, y en el frontend se ve por qué salió cada documento (palabras resaltadas).
- Puede responder "no hay resultados" (*EZLN* devuelve 1 documento; *UNACH*, 2), en lugar de rellenar con documentos que no tienen que ver.
- El índice ocupa poco y se actualiza agregando entradas al índice invertido, sin recalcular todo.

Desventajas

- Depende de que la consulta y el documento compartan palabras: si no las comparten, falla (*felino con manchas* = 0.431; en las notas cortas de la Tarea 3, 0 en las semánticas).
- Depende del preprocesamiento: los lemas de spaCy son la mitad del motor, y tardan 14 s en cargar.

- Coincidencias accidentales: en *temblor...* pone *Café de Chiapas* cuarto porque comparte *costa* y *Pacífico*.
- Con varios hilos procesa una consulta a la vez (GIL + spaCy); para escalar hay que usar varios procesos.

Embeddings mpnet (semántico)

Ventajas

- Entiende paráfrasis y sinónimos sin listas de sinónimos: en las notas cortas resolvió todas las semánticas, y aquí resuelve *mosquitos* (1.000) y *computadoras que aprenden* (0.860, mejor que BM25 con 0.730).
- Es multilingüe y no necesita lematizar ni quitar stopwords.
- Sigue siendo rápido con un usuario (25 ms), y su costo por consulta casi no depende del tamaño del corpus, porque el producto matriz–vector es barato (con millones de documentos se usaría un índice ANN como el HNSW de Qdrant de la Tarea 3).

Desventajas

- Fue el peor en este corpus (0.757): encuentra el tema, pero no el documento exacto (volcán en lugar de terremoto, guacamaya en lugar de quetzal, cocodrilo en lugar de jaguar).
- Falla con nombres propios que no conoce: *Montes Azules* → *Cascadas de Agua Azul*.
- Lee solo 128 tokens: con documentos largos hay que partirlos en pasajes, y el resultado cambia hasta 0.085 de nDCG según cómo se haga (Tabla 4).
- Nunca dice "no hay resultados". Siempre devuelve k documentos, aunque ninguno tenga que ver.
- Necesita GPU para escalar, y aun con GPU se satura en unas 50 peticiones/s. En el escenario mixto, su p50 subió a 2.2 s porque comparte la GPU con el híbrido.
- Es una caja negra: no se puede explicar por qué un coseno de 0.61 le gana a uno de 0.58.

Híbrido RRF + cross-encoder

Ventajas

- Empata con BM25 en calidad global (0.922; MRR = 0.961) y es el mejor en las consultas semánticas (0.868): usa lo mejor de los dos recuperadores.
- Es el más robusto ante consultas mezcladas: *zonas arqueológicas mayas en la selva* = 0.986 (BM25: 0.759; denso: 0.635).
- RRF no necesita calibrar escalas ni pesos (la lección de la Tarea 3), y el cross-encoder corrige los errores de la fusión, porque lee la consulta y el documento juntos.
- El costo del re-ranking depende de cuántos candidatos se reordenan (10), no del tamaño del corpus.

Desventajas

- Es el más caro: 93 ms con 1 usuario y 3.7 s con 50; se satura en 13.4 peticiones/s. Solo cumple p95 $<$ 500 ms hasta unos 5 usuarios.
- Necesita tener cargados tres modelos (spaCy, mpnet y el cross-encoder), con unos 45 s de arranque y $\approx$2.5 GB de memoria de GPU.
- El cross-encoder no siempre mejora: baja en cuatro consultas mixtas respecto a RRF. Además depende de qué texto se le pase: con el pasaje suelto reordenaba peor que con el documento completo.

- Hereda las limitaciones de la primera etapa: si el documento relevante no está en el top-20 de BM25 ni en el del denso, el cross-encoder no lo puede rescatar.
- Es el más difícil de depurar: tiene tres etapas, dos modelos y varios parámetros (profundidad, k de RRF, número de candidatos).

8. Conclusiones

1. Se integraron los tres motores de la unidad (BM25, embeddings mpnet e híbrido RRF + cross-encoder) en un sistema fullstack: una API REST en FastAPI con la misma interfaz para los tres y un frontend que permite compararlos lado a lado. El corpus son 60 artículos de Wikipedia obtenidos por web scraping (23 872 palabras, cada documento con al menos 140).

2. **Calidad:** BM25 y el híbrido empatan (nDCG@5 = 0.922) y los embeddings quedan atrás (0.757). En documentos largos la paráfrasis casi siempre comparte palabras con el texto, y la ventaja del denso que se vio con las notas cortas desaparece. El mejor motor depende del corpus.

3. **Latencia:** con 1 usuario, p50/p95 = 12/17 ms (BM25), 25/35 ms (embeddings) y 93/103 ms (híbrido). Con 50 usuarios: 259/305 ms, 958/1 106 ms y 3 671/3 742 ms. Las 5 600 peticiones respondieron sin errores.

4. **Escalabilidad:** los tres motores se saturan desde los 5 usuarios (190, 50 y 13 peticiones/s), porque el servidor procesa una consulta a la vez (GIL y candado de la GPU). A partir de ahí la latencia crece en línea recta con el número de usuarios, como predice la ley de Little. En carga mixta, el motor más lento retrasa a los que comparten la GPU con él.

5. **Decisión:** BM25 como motor por omisión y el híbrido como búsqueda profunda opcional. Para escalar el híbrido, lo siguiente sería agrupar consultas en la GPU (*batching*) y guardar en cache las consultas frecuentes.

A. Fuentes del corpus

Los 60 artículos se consultaron en `https://es.wikipedia.org` el 5 de noviembre de 2026 (licencia CC BY-SA 4.0). La columna *Rev.* es el identificador de la revisión descargada: la versión exacta se puede abrir con `https://es.wikipedia.org/w/index.php?oldid=<Rev.>`.

ID	Título	Categ.	Pal.	Rev.	URL
d01	Cañón del Sumidero	turismo	376	172643749	https://es.wikipedia.org/wiki/Ca%C3%B1%C3%B3n_del_Sumidero
d02	Zona arqueológica de Palenque	turismo	351	175327324	https://es.wikipedia.org/wiki/Zona_arqueol%C3%B3gica_de_Palenque
d03	San Cristóbal de Las Casas	turismo	384	175733748	https://es.wikipedia.org/wiki/San_Crist%C3%B3bal_de_las_Casas
d04	Parque nacional Lagunas de Montebello	turismo	394	167234872	https://es.wikipedia.org/wiki/Parque_nacional_Lagunas_de_Montebello
d05	Cascadas de Agua Azul	turismo	384	171359515	https://es.wikipedia.org/wiki/Cascadas_de_Agua_Azul
d06	Bonampak	turismo	528	175620989	https://es.wikipedia.org/wiki/Bonampak
d07	Yaxchilán	turismo	418	175621209	https://es.wikipedia.org/wiki/Yaxchil%C3%A1n
d08	Toniná	turismo	506	175904207	https://es.wikipedia.org/wiki/Tonin%C3%A1
d09	Chiapa de Corzo	turismo	356	175728086	https://es.wikipedia.org/wiki/Chiapa_de_Corzo
d10	Comitán de Domínguez	turismo	364	175445065	https://es.wikipedia.org/wiki/Comit%C3%A1n_de_Dom%C3%ADnguez

ID	Título	Categ.	Pal.	Rev.	URL
d11	Tuxtla Gutiérrez	turismo	405	175447129	https://es.wikipedia.org/wiki/Tuxtla_Guti%C3%A9rrez
d12	Tapachula de Córdova y Ordóñez	turismo	353	175697322	https://es.wikipedia.org/wiki/Tapachula
d13	Alouatta pigra	naturaleza	370	174362322	https://es.wikipedia.org/wiki/Alouatta_pigra
d14	Sima de las Cotorras	turismo	412	166436618	https://es.wikipedia.org/wiki/Sima_de_las_Cotorras
d15	Misol-Ha	turismo	366	174757733	https://es.wikipedia.org/wiki/Misol-Ha
d16	Izapa	turismo	418	175670930	https://es.wikipedia.org/wiki/Izapa
d17	Selva Lacandona	naturaleza	457	173199028	https://es.wikipedia.org/wiki/Selva_Lacandona
d18	Reserva de la biosfera Montes Azules	naturaleza	353	169449203	https://es.wikipedia.org/wiki/Reserva_de_la_biosfera_Montes_Azules
d19	Reserva de la Biósfera El Triunfo	naturaleza	393	174870141	https://es.wikipedia.org/wiki/Reserva_de_la_Biosfera_El_Triunfo
d20	Volcán Tacaná	naturaleza	393	175281558	https://es.wikipedia.org/wiki/Volc%C3%A1n_Tacan%C3%A1
d21	El Chichón	naturaleza	427	175383485	https://es.wikipedia.org/wiki/El_Chich%C3%B3n
d22	Río Grijalva	naturaleza	364	174220219	https://es.wikipedia.org/wiki/R%C3%ADo_Grijalva
d23	Río Usumacinta	naturaleza	392	174399923	https://es.wikipedia.org/wiki/R%C3%ADo_Usumacinta
d24	Pharomachrus mocinno	naturaleza	359	175778135	https://es.wikipedia.org/wiki/Pharomachrus_mocinno
d25	Panthera onca	naturaleza	445	175989251	https://es.wikipedia.org/wiki/Panthera_onca
d26	Tapirus bairdii	naturaleza	413	171881569	https://es.wikipedia.org/wiki/Tapirus_bairdii
d27	Ara macao	naturaleza	421	174715933	https://es.wikipedia.org/wiki/Ara_macao
d28	Crocodylus acutus	naturaleza	402	175498341	https://es.wikipedia.org/wiki/Crocodylus_acutus
d29	Terremoto de Chiapas de 2017	riesgos	360	175786428	https://es.wikipedia.org/wiki/Terremoto_de_Chiapas_de_2017
d30	Huracán Stan	riesgos	402	175442214	https://es.wikipedia.org/wiki/Hurac%C3%A1n_Stan
d31	Dengue	riesgos	412	175686007	https://es.wikipedia.org/wiki/Dengue
d32	Malaria	riesgos	371	175500442	https://es.wikipedia.org/wiki/Paludismo
d33	Deforestación	riesgos	374	175674306	https://es.wikipedia.org/wiki/Deforestaci%C3%B3n
d34	Sequía	riesgos	400	174691270	https://es.wikipedia.org/wiki/Sequ%C3%ADa
d35	Los Parachicos	cultura	453	172643617	https://es.wikipedia.org/wiki/Parachico
d36	Marimba	cultura	411	175214962	https://es.wikipedia.org/wiki/Marimba
d37	Pueblo tsotsil	cultura	447	176004492	https://es.wikipedia.org/wiki/Pueblo_tsotsil
d38	Pueblo tseltal	cultura	476	174943587	https://es.wikipedia.org/wiki/Pueblo_tseltal
d39	Lacandones	cultura	408	175570912	https://es.wikipedia.org/wiki/Lacandones
d40	Zoque (etnia)	cultura	367	174973747	https://es.wikipedia.org/wiki/Zoque_%28etnia%29
d41	Pox	cultura	393	168813802	https://es.wikipedia.org/wiki/Pox
d42	Ámbar de Chiapas	cultura	380	173245678	https://es.wikipedia.org/wiki/%C3%81mbar_de_Chiapas
d43	Ejército Zapatista de Liberación Nacional	cultura	353	175993169	https://es.wikipedia.org/wiki/Ej%C3%A9rcito_Zapatista_de_Liberaci%C3%B3n_Nacional
d44	Rosario Castellanos	cultura	420	176079894	https://es.wikipedia.org/wiki/Rosario_Castellanos
d45	Jaime Sabines	cultura	494	175938551	https://es.wikipedia.org/wiki/Jaime_Sabines
d46	Tascalate	cultura	140	172643697	https://es.wikipedia.org/wiki/Tascalate
d47	Café de Chiapas	economia	629	172909337	https://es.wikipedia.org/wiki/Caf%C3%A9_de_Chiapas
d48	Theobroma cacao	economia	381	175801968	https://es.wikipedia.org/wiki/Theobroma_cacao
d49	Chicoasén	economia	162	175292347	https://es.wikipedia.org/wiki/Chicoas%C3%A9n
d50	Presa La Angostura	economia	354	171360069	https://es.wikipedia.org/wiki/Presa_La_Angostura
d51	Tren Maya	economia	398	175293915	https://es.wikipedia.org/wiki/Tren_Maya
d52	Soconusco	economia	356	176072615	https://es.wikipedia.org/wiki/Soconusco
d53	Universidad Autónoma de Chiapas	tecnologia	414	173186861	https://es.wikipedia.org/wiki/Universidad_Aut%C3%B3noma_de_Chiapas
d54	Inteligencia artificial	tecnologia	424	176155933	https://es.wikipedia.org/wiki/Inteligencia_artificial
d55	Aprendizaje automático	tecnologia	386	176151143	https://es.wikipedia.org/wiki/Aprendizaje_autom%C3%A1tico
d56	Minería de datos	tecnologia	472	175301843	https://es.wikipedia.org/wiki/Miner%C3%ADa_de_datos
d57	Procesamiento de lenguajes naturales	tecnologia	485	176063044	https://es.wikipedia.org/wiki/Procesamiento_de_lenguajes_naturales
d58	Motor de búsqueda	tecnologia	386	176155161	https://es.wikipedia.org/wiki/Motor_de_b%C3%BAsqueda
d59	Robótica	tecnologia	354	176159493	https://es.wikipedia.org/wiki/Rob%C3%B3tica

ID	Título	Categ.	Pal.	Rev.	URL
d60	Red neuronal artificial	tecnologia	406	175719671	https://es.wikipedia.org/wiki/Red_neuronal_artificial

B. Reproducir los resultados

```
pip install -r requirements.txt && python -m spacy download es_core_news_sm
python scraping/scrape_wikipedia.py # corpus (ya incluido en data/)
uvicorn app:app --app-dir backend --port 8000 # API + frontend
python evaluacion/evaluar_calidad.py # tabla de calidad
python evaluacion/ablacion_denso.py # ablacion del motor denso
python carga/prueba_carga.py # prueba de carga (con el servidor arriba)
python carga/graficar_carga.py # graficas de latencia
```